

Getting Started & Local Development

▼ Relevant source files

Backend/STUDER/.mvn/wrapper/maven-wrapper.properties

Backend/STUDER/Dockerfile

Backend/STUDER/mvnw

Backend/STUDER/pom.xml

Backend/STUDER/src/main/resources/application.properties

Docker/docker-compose.yml

Frontend/STUDER/angular.json

This page provides a comprehensive technical guide for setting up and running the STUDER platform in a local development environment. It covers the containerized infrastructure, environment configuration, and specific workflows for backend and frontend development.

System Infrastructure Overview

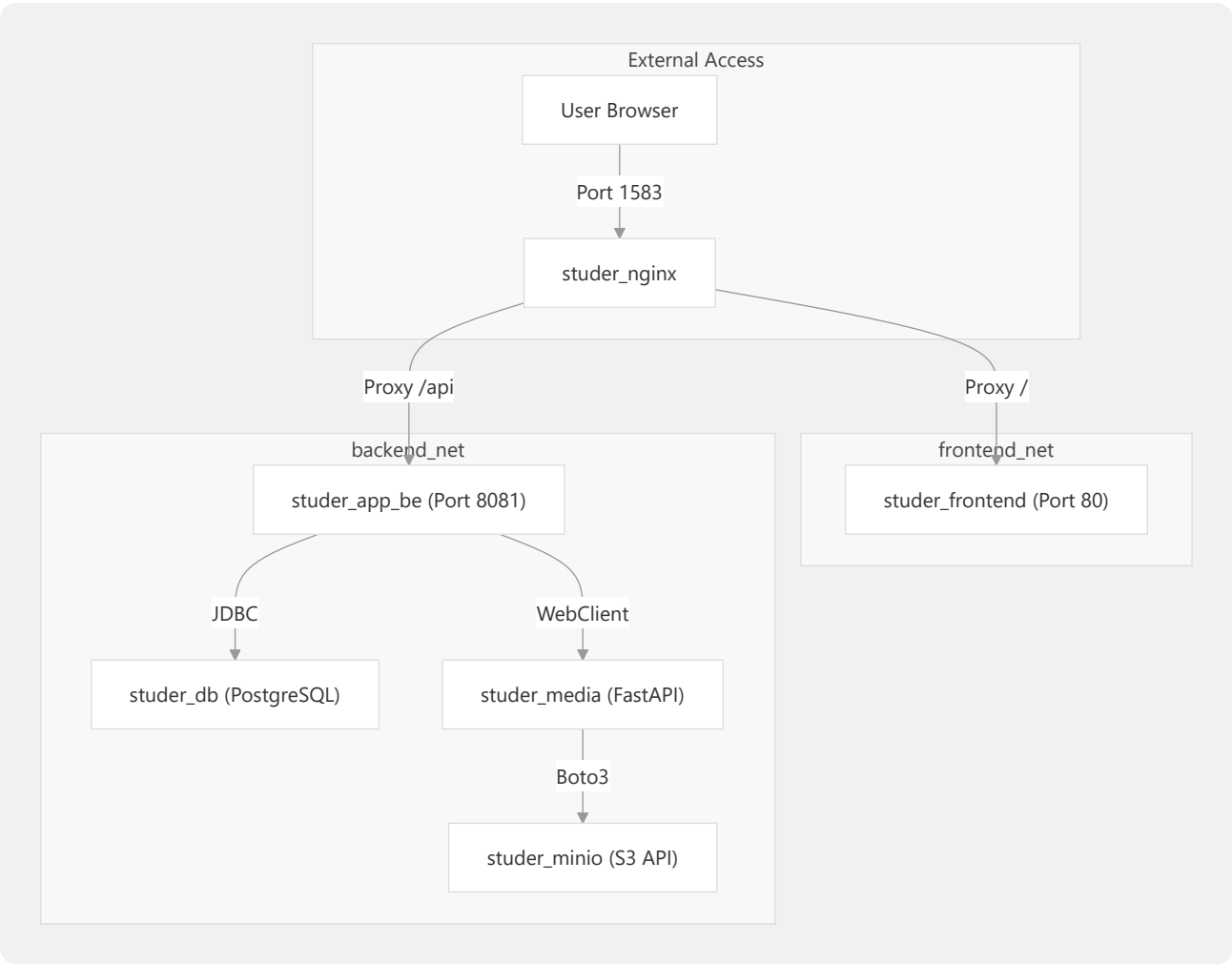
STUDER utilizes a microservices-oriented architecture managed via Docker Compose. The infrastructure is partitioned into two distinct networks to isolate internal service communication from public-facing traffic.

Network Topology and Service Discovery

The system defines two bridge networks:

- **frontend_net** : Connects the Nginx reverse proxy and the Angular frontend
Docker/docker-compose.yml 91-92
- **backend_net** : Connects the Spring Boot backend, FastAPI media service, PostgreSQL database, and MinIO storage
Docker/docker-compose.yml 93-94

Service Interaction Diagram



Sources: `Docker/docker-compose.yml` 1-94

`Backend/STUDER/src/main/resources/application.properties` 25

Environment Configuration

Configuration is driven by environment variables defined in a `.env` file (referenced in `docker-compose.yml`) and Spring properties.

Required Environment Variables

The following variables must be defined for the containers to initialize correctly:

Variable	Description	Component
<code>POSTGRES_DB</code>	Database name	<code>studer_db</code> , <code>studer_app_be</code>
<code>POSTGRES_USER</code>	Database administrative user	<code>studer_db</code> , <code>studer_app_be</code>
<code>POSTGRES_PASSWORD</code>	Database user password	<code>studer_db</code> , <code>studer_app_be</code>

Variable	Description	Component
<code>JWT_SECRET</code>	HS256 key for signing tokens	<code>studer_app_be</code>
<code>MINIO_ROOT_USER</code>	Access key for MinIO	<code>studer_minio</code> , <code>studer_media</code>
<code>MINIO_ROOT_PASSWORD</code>	Secret key for MinIO	<code>studer_minio</code> , <code>studer_media</code>

Sources: `Docker/docker-compose.yml` 7-83

`Backend/STUDER/src/main/resources/application.properties` 6

Backend Development (Spring Boot)

The backend is a Java 17 Spring Boot application `Backend/STUDER/pom.xml` 30 It supports two primary execution modes: containerized (production-like) and local (development).

Maven Wrapper (`mvnw`)

Developers should use the included Maven Wrapper to ensure build consistency without requiring a local Maven installation `Backend/STUDER/mvnw` 1-151

- **Configuration:** Defined in `.mvn/wrapper/maven-wrapper.properties`
`Backend/STUDER/.mvn/wrapper/maven-wrapper.properties` 1-4
- **Usage:** Run `./mvnw clean install` from the `Backend/STUDER` directory.

Spring Profiles

The application behavior changes based on the `SPRING_PROFILES_ACTIVE` variable:

- **dev:** Default profile in `application.properties`
`Backend/STUDER/src/main/resources/application.properties` 7 Enables `spring-boot-devtools` `Backend/STUDER/pom.xml` 51-54 and typically uses H2 or a local DB.
- **prod:** Set in `docker-compose.yml` `Docker/docker-compose.yml` 23 Configures the app to connect to the `db` container and enforces stricter security settings.

Build and Execution Flow

The `Dockerfile` utilizes a multi-stage build to minimize the production image size:

1. **Build Stage:** Uses `maven:3.9.6-eclipse-temurin-17-alpine` to compile and package the JAR, skipping tests `Backend/STUDER/Dockerfile` 2-6
2. **Runtime Stage:** Uses `eclipse-temurin:17-jre-alpine` to run the resulting `app.jar` `Backend/STUDER/Dockerfile` 9-13

Sources: `Backend/STUDER/pom.xml` 1-154 `Backend/STUDER/Dockerfile` 1-14
`Backend/STUDER/src/main/resources/application.properties` 1-26

Frontend Development (Angular)

The frontend is an Angular 19 application `Frontend/STUDER/angular.json` 6

Local Development Server

For rapid development, use the Angular CLI:

- **Command:** `ng serve` or `npm start`.
- **Configuration:** Uses the `development` block in `angular.json`, which disables optimization and enables source maps `Frontend/STUDER/angular.json` 56-60
- **Port:** Defaults to `4200`.

Production Build

When running via Docker, the frontend is compiled and served by an internal server (or Nginx in the `frontend` container):

- **Output Path:** `dist/studer` `Frontend/STUDER/angular.json` 16
- **Optimization:** Production builds enable `outputHashing` and strict bundle budgets `Frontend/STUDER/angular.json` 41-55

Sources: `Frontend/STUDER/angular.json` 1-106

Local Setup Steps

1. Database and Storage Bootstrap

Run the infrastructure components first to ensure the backend can connect:

```
docker compose up -d db minio
```

Sources: Docker/docker-compose.yml 3-73

2. Media Service Setup

The media service depends on MinIO Docker/docker-compose.yml 42-43 It exposes an API on port 8000 for image processing Docker/docker-compose.yml 40

```
docker compose up -d studer-media
```

3. Backend Execution

You can run the backend within Docker or via your IDE. If running in an IDE, ensure you provide the `JWT_SECRET` environment variable. The backend communicates with the media service via the `app.media.service.url` property

Backend/STUDER/src/main/resources/application.properties 25

4. Reverse Proxy Access

The Nginx container maps port `1583` on the host to the internal network

Docker/docker-compose.yml 68 This is the primary entry point for testing the full integration.

Code Entity Mapping: Infrastructure to Configuration

Sources: Docker/docker-compose.yml 1-30

Backend/STUDER/src/main/resources/application.properties 6-7

Frontend/STUDER/angular.json 13-16

Troubleshooting Local Development

- **JWT Errors:** Ensure `JWT_SECRET` is consistent across the `.env` file and any local IDE run configurations Backend/STUDER/src/main/resources/application.properties 6
- **CORS Issues:** Check `app.cors.allowed-origins` in `application.properties`. By default, it allows `localhost:4200` and `localhost:1583`

Backend/STUDER/src/main/resources/application.properties 20

- **Database Connectivity:** In **prod** profile, the backend expects the host **db**
Docker/docker-compose.yml 27 If running the backend locally (outside Docker), change the host to **localhost**.

Sources: Backend/STUDER/src/main/resources/application.properties 19-20

Docker/docker-compose.yml 22-29